

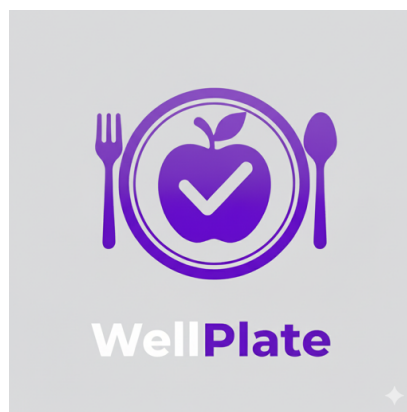


POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

WellPlate: Design Document

Authors: **Jacopo Piazzalunga, Gabriele Puglisi, Davide Salonico**



Academic Year: 2024-2025

Contents

Contents	i
----------	---

1	Introduction	1
1.1	Purpose	1
1.2	Requirements	2
1.2.1	Functional Requirements	2
1.2.2	Non-Functional Requirements	3
1.3	Features Implemented	3
1.3.1	User Registration and Authentication	3
1.3.2	Profile Setup and Personalization	3
1.3.3	AI-Powered Meal Plan Generation	3
1.3.4	Meal Plan Management	4
1.3.5	Daily Meal Tracking	4
1.3.6	Multi-Device Synchronization	4
1.3.7	Subscription Management	4
1.3.8	Nutritionist Consultation System	4
1.3.9	Professional Nutritionist Interface	4
1.3.10	Chat Communication	5
1.3.11	Real-Time Notifications	5
2	Application Architecture	6
2.1	Overview	6
2.2	Data Model	7
2.2.1	Core User Management	7
2.2.2	Meal Planning Domain	8
2.2.3	Local Caching Layer	10

2.2.4	Progress Tracking and Daily Views	12
2.2.5	Chat and Communication System	13
2.2.6	Data Architecture Patterns	13
2.3	Data Management	14
2.3.1	Amazon DynamoDB	14
2.3.2	AWS AppSync	14
2.3.3	Isar Local Database	14
2.3.4	Shared Preferences	15
2.3.5	AWS Cognito + Google Sign-In	15
2.4	Notification Implementation	15
2.5	External Services	15
2.5.1	Amazon S3	15
2.5.2	Google Gemini 2.5 Flash	16
2.5.3	OpenStreetMap Integration and Geocoding	16
2.6	Dependencies	16
2.7	Widget Architecture	17
2.7.1	Responsive Design Strategy	17
2.7.2	Widget Organization	17
2.8	Sequence Diagrams	18
2.8.1	User Meal Plan Generation	18
2.8.2	Nutritionist Validation Request Flow	20
2.8.3	Real-time Chat Message Flow	21
2.8.4	Daily Meal Completion Tracking Flow	22
3	User Interface (UI)	23
3.1	UI Design Choices	23
3.1.1	Design Philosophy	23
3.1.2	Navigation Pattern	23
3.2	Smartphone UI	24
3.2.1	Authentication and Role Selection	24
3.2.2	User Interface – Today Tab (Daily Dashboard)	25
3.2.3	User Interface – My Plans Tab (Meal Plan Management)	26
3.2.4	User Interface – Settings Tab (Profile Management)	27
3.2.5	Nutritionist Interface – Validate Plans Tab (Professional Dashboard)	30
3.2.6	Nutritionist Interface – Settings Tab (Professional Profile)	32

3.2.7	Chat System (Shared Communication Interface)	32
3.2.8	Meal Plan Generation Interface	35
3.2.9	Detailed Meal and Plan Viewing	37
3.2.10	Tablet UI	37
3.3	Accessibility	41
4	Testing Campaign	42
4.1	Testing Environment	42
4.2	Unit Testing	42
4.2.1	Coverage Report	43
4.3	User Testing	43
4.3.1	Testing Approach	43
4.3.2	Key Questions for Feedback	44
5	Future Works	45
5.1	Improve AI integration	45
5.2	Wearable Device Integration	45
5.3	Social and Community Features	45
5.4	Advanced Nutritionist Tools	46
5.5	Technical Improvements	46

1 | Introduction

1.1. Purpose

WellPlate is a multi-platform mobile application designed to revolutionize personal nutrition planning through AI-powered meal generation and professional nutritionist consultation. The idea was born from recognizing the limitations of existing meal planning applications, which often lack sufficient personalization and intelligent adaptation to individual needs. Most current solutions provide generic meal plans that don't adequately consider the complex interplay of personal factors such as lifestyle, dietary restrictions, exercise habits, and health goals. WellPlate addresses this gap by leveraging Large Language Model (LLM) technology to generate highly personalized meal plans and providing optional access to certified nutritionists for professional guidance. The target audience includes health-conscious individuals aged 15 to 55 who seek convenient, personalized nutrition planning that adapts to their specific needs and lifestyle requirements. This application makes it easier for people to start eating well and following correct nutrition plans without the complexity typically associated with meal planning. This document explains the most important design choices we made and the motivations behind them.

1.2. Requirements

1.2.1. Functional Requirements

The following list contains the functional requirements that the application should satisfy.

- FR1 Users should be able to sign in/sign up in the app using email or Google SSO
- FR2 Nutritionists should be able to sign in/sign up in using email (no Google SSO at first login)
- FR3 Nutritionists should be able to provide detailed professional information during registration
- FR4 Both users and nutritionists should be able to logout from the app
- FR5 Registered users should be able to edit their personal profile information
- FR6 Registered users should be able to input lifestyle data (age, sex, exercise level)
- FR7 Registered users should be able to specify diet preferences and restrictions
- FR8 Registered users should be able to add custom dietary requests in free text
- FR9 Registered users should be able to generate AI-powered personalized meal plans
- FR10 Registered users should be able to view detailed recipes with ingredients and instructions
- FR11 Registered users should be able to manage multiple meal plans simultaneously
- FR12 Registered users should be able to select and activate a meal plan
- FR13 Registered users should be able to track daily meal consumption
- FR14 Registered users should be able to monitor calorie intake throughout the day
- FR15 Registered users should be able to access meal plans across multiple devices
- FR16 Registered users should be able to subscribe to premium features
- FR17 Registered users should be able to see nutritionists' addresses on maps (when provided)
- FR18 Subscribed users should be able to request nutritionist validation of meal plans
- FR19 Subscribed users should be able to chat with assigned nutritionists
- FR20 Nutritionists should be able to review and validate meal plans
- FR21 Nutritionists should be able to modify meal plans during review

FR22 Nutritionists should be able to approve or reject meal plans

FR23 Nutritionists should be able to manage their availability status

FR24 Nutritionists should be able to access their profile and consultation history

1.2.2. Non-Functional Requirements

- **Usability:** The interface should be intuitive for users with basic smartphone/tablet experience
- **Security:** User will not be able to inject malicious prompts when requesting the meal plans.
- **Compatibility:** The app should support iOS 14+, Android 8+, and iPadOS 14+
- **Accessibility:** Dark and White Theme version with italian and english translations available

1.3. Features Implemented

The functionalities implemented in the app, following a typical user interaction flow, are:

1.3.1. User Registration and Authentication

Users can sign in or sign up using email address or Single-Sign-On (SSO) functionality with Google identity providers. A password reset protocol is implemented for email-based accounts.

1.3.2. Profile Setup and Personalization

Registered users can input comprehensive personal information including age, sex, weight, height, meal preferences (number of daily meals), and exercise frequency (sessions per week). Users can select from 14 standardized allergen categories including gluten, crustaceans, eggs, fish, peanuts, soybeans, milk, nuts, celery, mustard, sesame seeds, sulphites, lupin, and molluscs. Additionally, users can specify dietary restrictions and dietary preferences in free text for complete personalization.

1.3.3. AI-Powered Meal Plan Generation

The application uses a hosted Large Language Model to generate personalized weekly meal plans based on user profile data. Each meal plan includes detailed recipes with complete ingredient lists and step-by-step cooking instructions, ensuring the plan accommodates all user preferences and requirements.

1.3.4. Meal Plan Management

Users can manage multiple meal plans simultaneously, each with different statuses: pending (awaiting generation), generated (ready for use), requesting validation (for premium users), validated (approved by nutritionist), rejected (requires discussion), or failed (generation error with explanation). Users can select one active meal plan for daily tracking.

1.3.5. Daily Meal Tracking

Users can view their daily meal schedule from the active meal plan and mark meals as consumed throughout the day. The application tracks calorie intake and provides progress monitoring for nutritional goals.

1.3.6. Multi-Device Synchronization

Meal plans are stored both locally on the device and in the cloud, providing seamless synchronization across multiple devices when users log in with the same account.

1.3.7. Subscription Management

Users can subscribe to premium features that unlock access to professional nutritionist consultations and meal plan validation services.

1.3.8. Nutritionist Consultation System

Premium users can request meal plan validation from available nutritionists. The system provides a list of certified nutritionists with profiles, availability status, and optionally an address. Users can initiate consultations and communicate through an integrated chat system.

1.3.9. Professional Nutritionist Interface

Nutritionists have dedicated functionality to review meal plans, make modifications when necessary, approve or reject plans, and communicate with users through the chat system. They can manage their availability status and address and access their consultation history.

1.3.10. Chat Communication

The application provides persistent chat functionality between users and nutritionists. While chat history remains accessible, active messaging requires an active premium subscription.

1.3.11. Real-Time Notifications

The application delivers real-time notifications to keep users informed while navigating different screens. Users receive notifications for meal plan status changes (generating, generated, requested validation, validated, rejected, failed) and new chat messages. Nutritionists receive notifications for new chat messages and incoming meal plan validation requests, ensuring timely responses to user consultations.

2 | Application Architecture

2.1. Overview

We have decided to build [WellPlate](#) using Flutter for the frontend and a serverless AWS backend architecture. Flutter is an open source framework developed by Google for building natively compiled and multiplatform applications from a single codebase. Dart is the language on which Flutter is based, optimized for fast apps on any platform.

The application follows a modern serverless architecture pattern with clear separation between frontend and backend services:

- [Frontend](#): The Flutter application uses AWS Amplify for seamless backend integration, providing APIs for authentication, data access, and real-time subscriptions.
- [Backend](#): A serverless architecture built with AWS CDK (Cloud Development Kit) that leverages multiple AWS services for scalability and reliability.

For the development of the application, different services are used, which can be divided into three categories:

- [Storage Services](#): APIs related to local and cloud storage of application data.
- [External Services](#): APIs related to functionalities and data provided by external services.

The integration of these services is implemented by exploiting asynchronous communication protocols when possible that, with the help of Riverpod's `AsyncNotifierProvider` pattern, allows the application to be as responsive and smooth as possible regardless of the performance of external services.

The internal structure of the app follows a [clean architecture](#) pattern with clear separation between presentation, business logic, and data layers. This pattern allows transparent replacement of external

services without affecting other parts of the application.

For device services, Flutter libraries help us keep the application device-independent by exploiting wrappers that permit the use of Application Programming Interfaces (API) regardless of the device used.

2.2. Data Model

Data shown in the application is structured using models that correspond to the GraphQL schema defined in our AWS AppSync API. The architecture employs a sophisticated dual-layer approach with server-side models for cloud operations and client-side cache models for offline functionality.

The data model is organized into five main domains, each addressing specific aspects of the nutrition planning and consultation system.

2.2.1. Core User Management

This domain handles user authentication, profiles, and subscription management for both regular users and nutrition professionals. The `UserDetails` entity contains comprehensive user profile information including physical attributes (weight, height), dietary preferences, allergies, and exercise frequency. `NutritionistProfile` manages professional information including specializations, availability status, and profile details. The subscription system differentiates between free and premium users through the `UserSubscriptionStatus` entity.

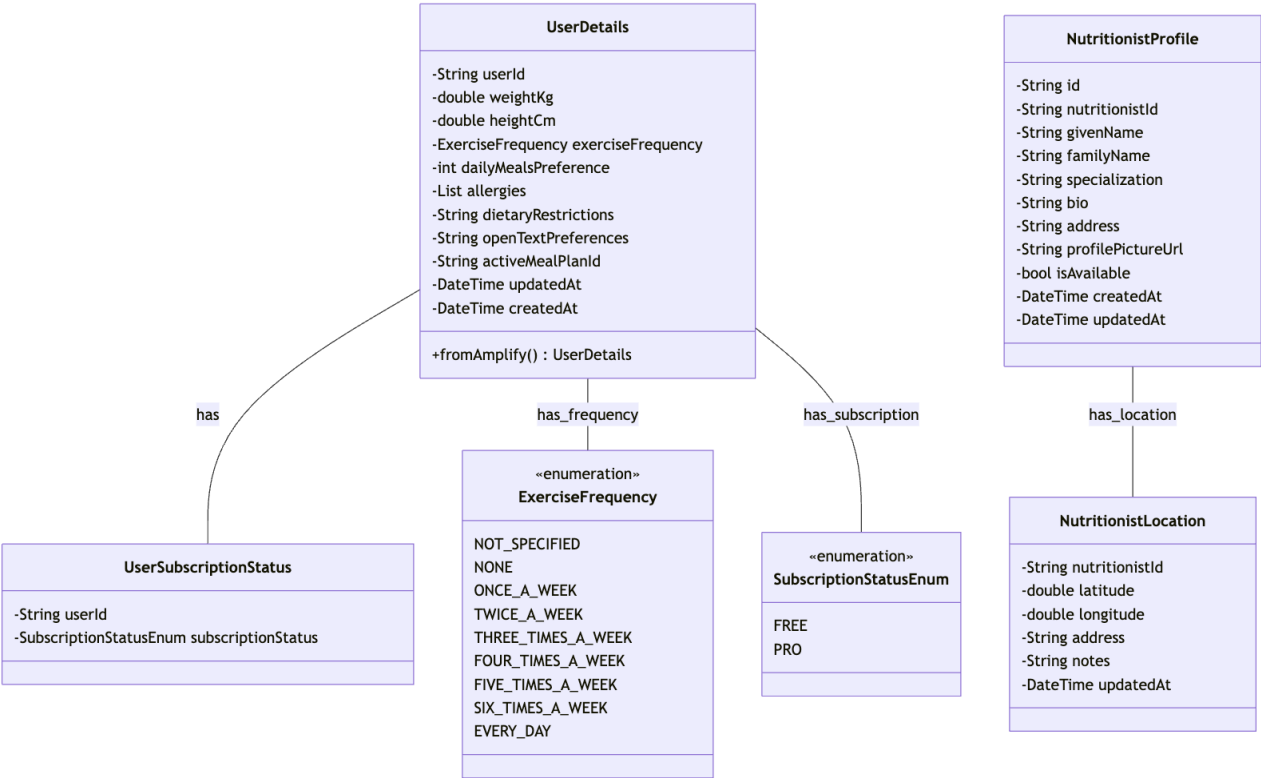


Figure 2.1: Core User Management Class Diagram

2.2.2. Meal Planning Domain

The core meal planning entities follow a hierarchical structure from complete meal plans down to individual nutritional components. `MealPlan` represents the top-level entity containing weekly meal structures, validation status, and nutritionist assignments. `DailyPlanData` organizes meals by weekday, while individual `Meal` entities contain recipe information, ingredient lists, and nutritional data. Each `Ingredient` includes detailed macronutrient information through the `Macros` entity, enabling precise nutritional tracking and analysis.

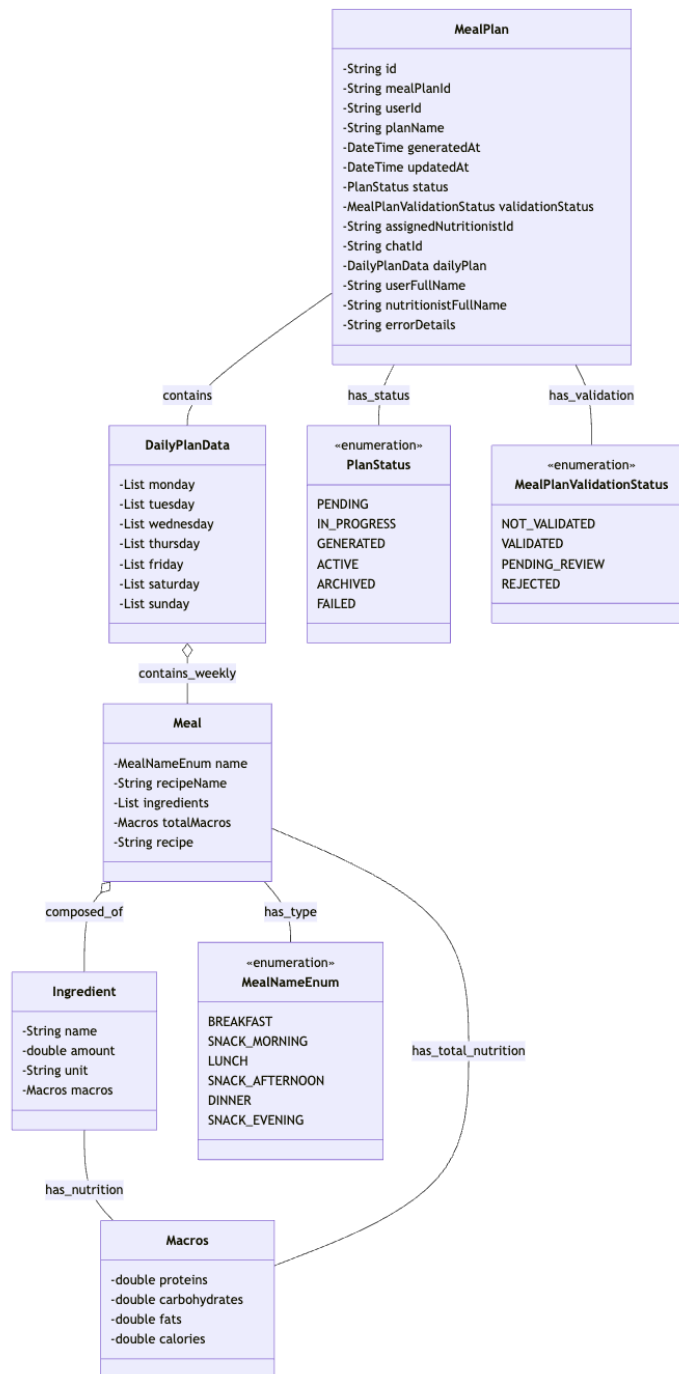


Figure 2.2: Meal Planning Domain Class Diagram

2.2.3. Local Caching Layer

The caching layer mirrors server-side models with additional functionality for offline operations and data synchronization. Each cache model (`MealPlanCache`, `MealCache`, `IngredientCache`, `MacrosCache`) includes conversion methods for seamless transformation between server and client representations. These models support JSON serialization for local storage and include timestamp tracking for data freshness validation.

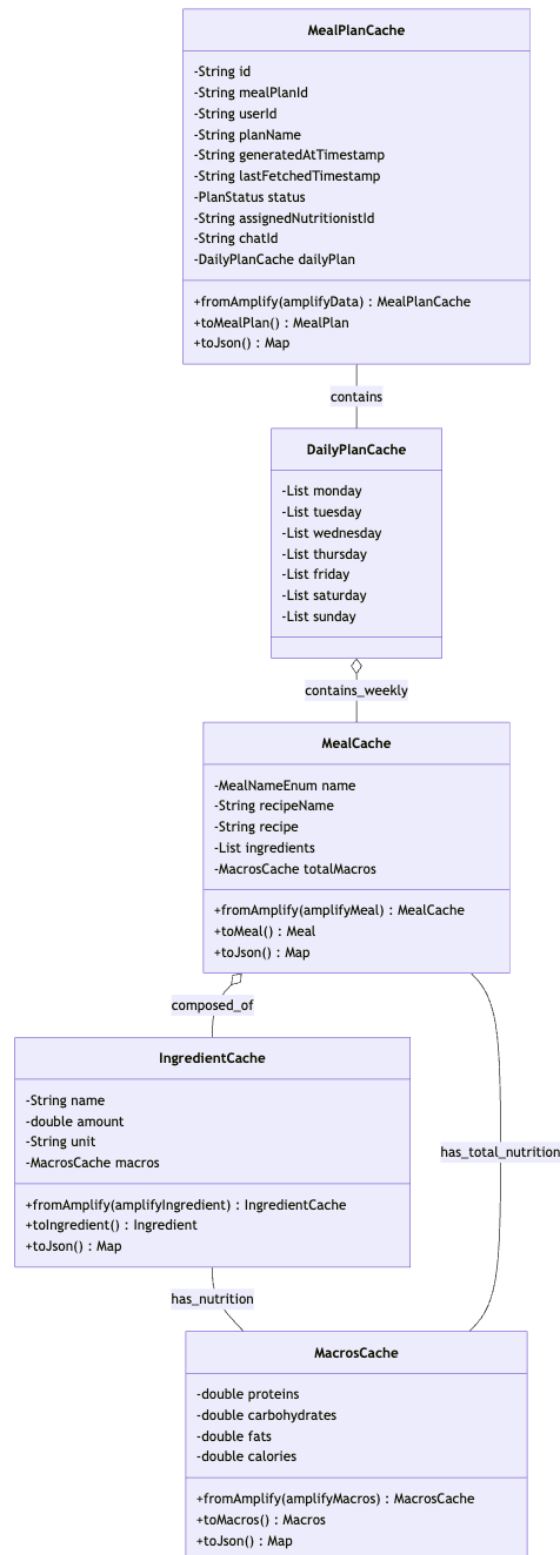


Figure 2.3: Local Caching Layer Class Diagram

2.2.4. Progress Tracking and Daily Views

This domain manages daily meal completion tracking and provides aggregated views for user progress monitoring. `TodaysPlan` combines active meal plan details with daily completion status, while `PlanDayCompletion` tracks which meals users have consumed throughout the day. `MealWithStatus` wraps individual meals with completion flags for real-time progress updates.

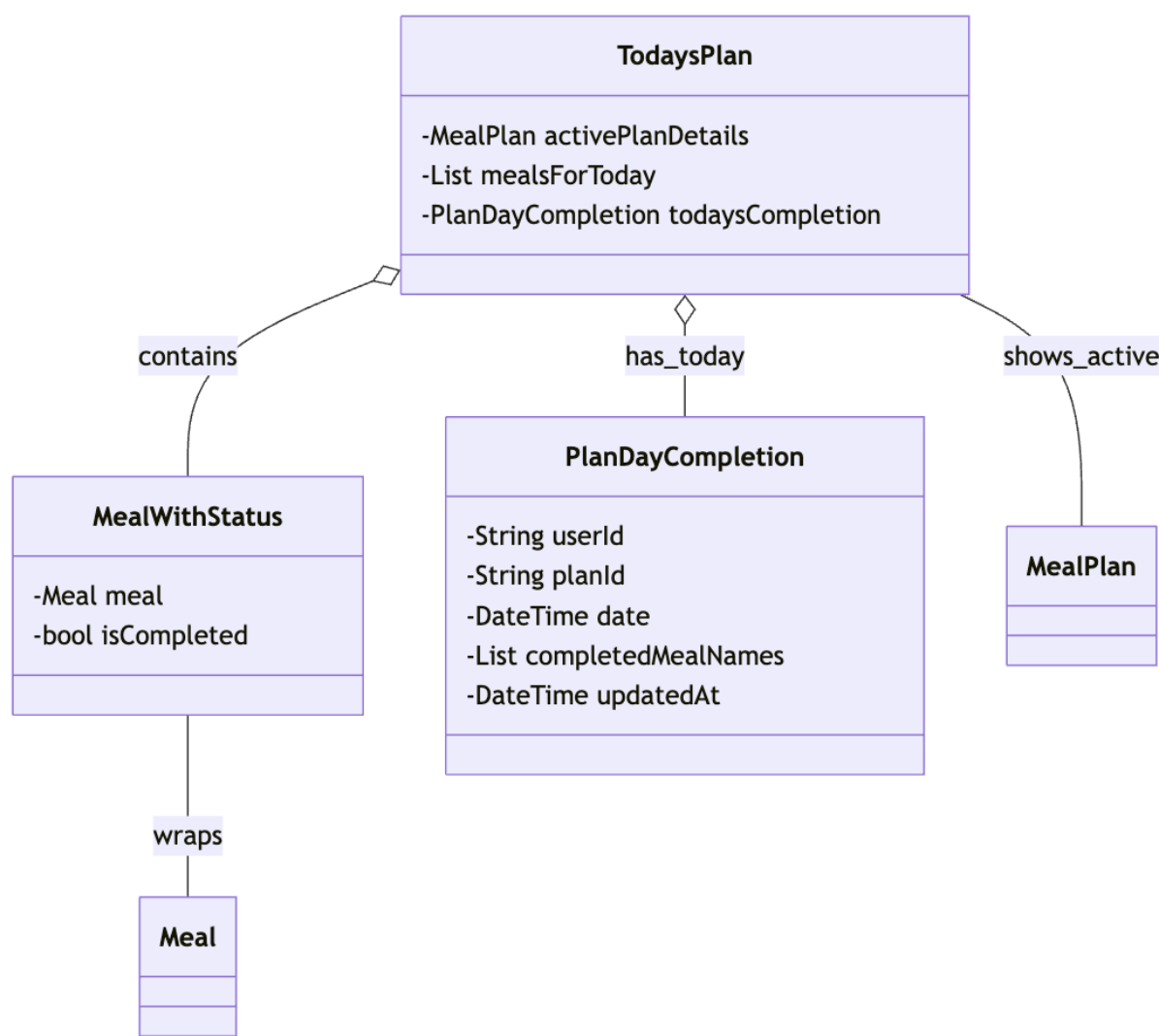


Figure 2.4: Progress Tracking and Daily Views Class Diagram

2.2.5. Chat and Communication System

The communication domain enables real-time messaging between users and nutritionists for consultation and meal plan validation discussions. `ChatMetadata` manages conversation sessions with participant information and unread message counts, while `ChatMessage` handles individual message content with sender identification and timestamps.

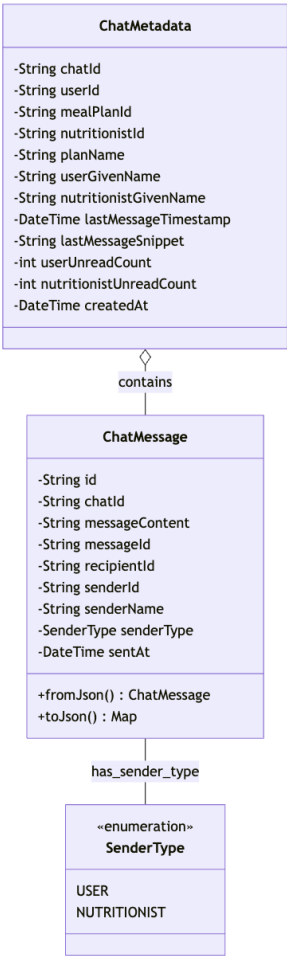


Figure 2.5: Chat and Communication System Class Diagram

2.2.6. Data Architecture Patterns

The data model implements several key architectural patterns:

- **Dual-Layer Architecture:** Some core entity exists in both server-side (AWS AppSync/DynamoDB) and client-side (Isar) forms, with conversion methods enabling seamless data flow between lay-

ers.

- **Hierarchical Composition:** Meal plans are structured hierarchically from weekly plans down to individual nutritional components, allowing both aggregate and detailed nutritional analysis.
- **Real-time Synchronization:** Chat messages and meal plan status updates use GraphQL subscriptions for immediate data synchronization across devices.
- **Offline-First Design:** Cache models include JSON serialization methods and timestamp tracking to support offline functionality with eventual consistency when connectivity returns.

Data is fetched from AWS AppSync using GraphQL queries and mutations, then parsed into Dart models for use throughout the application.

2.3. Data Management

2.3.1. Amazon DynamoDB

Amazon DynamoDB serves as the primary NoSQL database for all application data. The database stores user profiles, meal plans, chat messages, nutritionist information, and meal completion tracking. DynamoDB provides automatic scaling, low-latency performance, and seamless integration with other AWS services.

2.3.2. AWS AppSync

AWS AppSync provides the GraphQL API layer that handles all client-server communication. It manages real-time subscriptions for chat messages and meal plan status updates, query resolution through Lambda functions, and automatic data synchronization across devices.

2.3.3. Isar Local Database

Isar serves as the local database solution for offline functionality and caching. It stores data like the active meal plan to ensure the application remains functional when offline. The local database synchronizes with DynamoDB when connectivity is restored, providing a seamless offline-first experience.

2.3.4. Shared Preferences

Shared preferences handle simple key-value storage for user settings and application preferences, such as theme selection, notification preferences, and cached authentication tokens.

2.3.5. AWS Cognito + Google Sign-In

AWS Cognito provides comprehensive user authentication and authorization services. It supports both email/password authentication and Google Single Sign-On integration. Cognito manages user groups (`USERS` and `NUTRITIONISTS`) and provides JWT tokens for secure API access. Users can register with Google SSO for convenience, while nutritionists register with detailed professional information using email authentication.

2.4. Notification Implementation

WellPlate implements real-time notifications through AWS AppSync subscriptions. The application subscribes to two main events:

- **Meal Plan Status Changes:** Users receive real-time updates when meal plans transition between states (pending, generating, generated, validated, rejected, failed).
- **Chat Messages:** Both users and nutritionists receive instant notifications for new messages in their conversations.

These subscriptions maintain persistent connections while the app is active, providing immediate updates without requiring background processing or external notification services. The real-time nature ensures users stay informed about meal plan progress and can respond promptly to nutritionist communications.

2.5. External Services

2.5.1. Amazon S3

Amazon S3 is used for nutritionist profile pictures ensuring secure and direct uploads from the mobile application.

2.5.2. Google Gemini 2.5 Flash

The Google Gemini 2.5 Flash model powers the AI-driven meal plan generation. Integration occurs through AWS Lambda functions within Step Function workflows, ensuring scalable and reliable meal plan creation. The AI processes user preferences, dietary restrictions, and nutritional requirements to generate personalized weekly meal plans with detailed recipes and nutritional information.

2.5.3. OpenStreetMap Integration and Geocoding

The `flutter_map` package is employed to provide an interactive map interface within WellPlate. This feature enables users to visualize the locations of nutritionists' offices directly on the map, enhancing spatial awareness and accessibility. The integration relies on OpenStreetMap as the underlying mapping service and supports redirection to external navigation applications. This is particularly valuable when users need driving directions, estimated travel times, or distance checks.

Furthermore, to streamline the registration process for nutritionists, WellPlate implements automatic geocoding. By leveraging the device's current location, the system can infer and pre-fill address information, thereby reducing manual input, saving time, and improving the overall user experience (UX).

2.6. Dependencies

The most significant Flutter packages and services included in the application are shown in Table 2.1.

Package/Service	Purpose
amplify_flutter	AWS Amplify integration for API, Auth, and Storage
amplify_api	GraphQL API client for AWS AppSync
amplify_auth_cognito	AWS Cognito authentication
amplify_authenticator	Prebuilt UI components for user sign-up, sign-in, and password reset
amplify_storage_s3	Uploading, downloading, and managing files in Amazon S3
riverpod	State management with AsyncNotifierProvider pattern
isar	Local NoSQL database for offline functionality
flutter_riverpod	Riverpod integration for Flutter widgets
go_router	Declarative routing and navigation
shared_preferences	Simple key-value storage for user preferences
cached_network_image	Efficient image loading and caching
geocoding	Converting geographic coordinates into addresses
intl	Internationalization and localization

Table 2.1: Main Flutter dependencies used in WellPlate

2.7. Widget Architecture

The application implements a responsive design framework to optimize user experience across different screen sizes, particularly smartphones and tablets. The widget architecture follows Flutter’s compositional pattern with reusable components.

2.7.1. Responsive Design Strategy

- **Smartphone Layout:** Single-column layouts optimized for portrait orientation with bottom navigation and drawer-based menu systems.
- **Tablet Layout:** Multi-column layouts that display related information side-by-side, such as meal plan lists with detailed views, and chat conversations with participant information.

2.7.2. Widget Organization

- **Page Widgets:** Top-level screens that coordinate multiple smaller widgets.
- **Feature Widgets:** Reusable components for specific functionality (meal cards, chat bubbles, nutrition charts).

- **Responsive Widgets:** Components that adapt their layout based on screen size using `MediaQuery` and `LayoutBuilder`.

The responsive design is achieved through `LayoutBuilder` widgets that dynamically select appropriate layouts based on screen dimensions, ensuring optimal usability across device types.

2.8. Sequence Diagrams

2.8.1. User Meal Plan Generation

These diagrams shows the complete AI-powered meal plan creation workflow, from initial user request through the AWS Step Function orchestration to final delivery via real-time subscriptions.

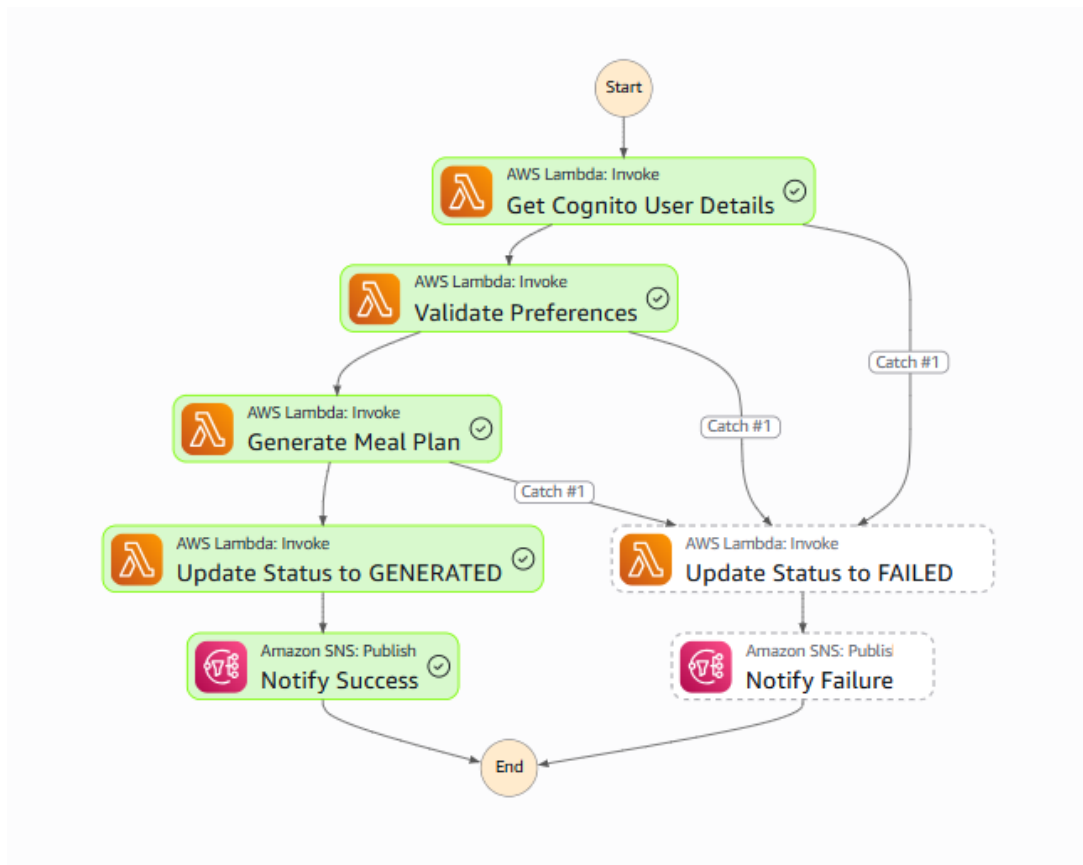
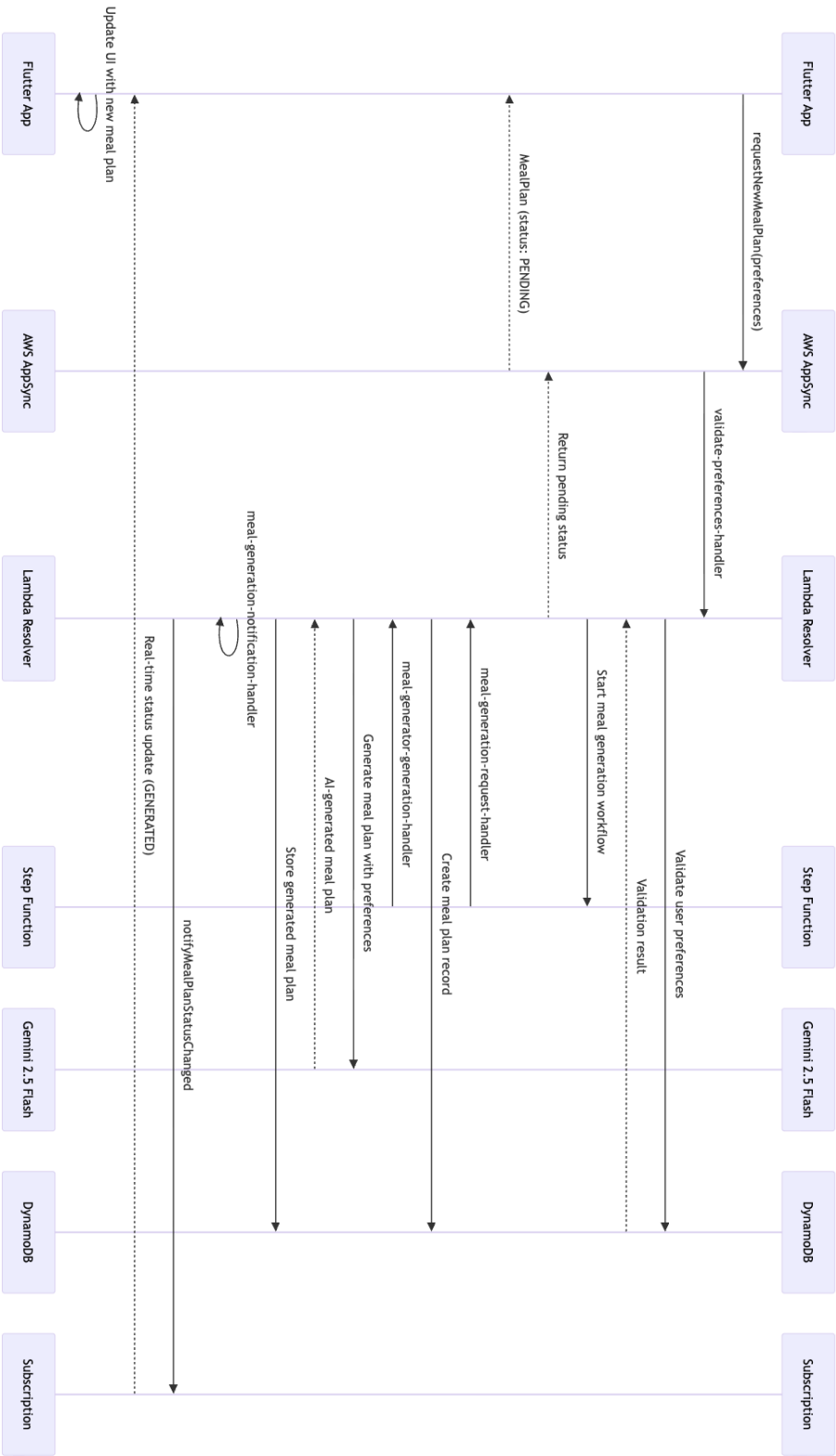
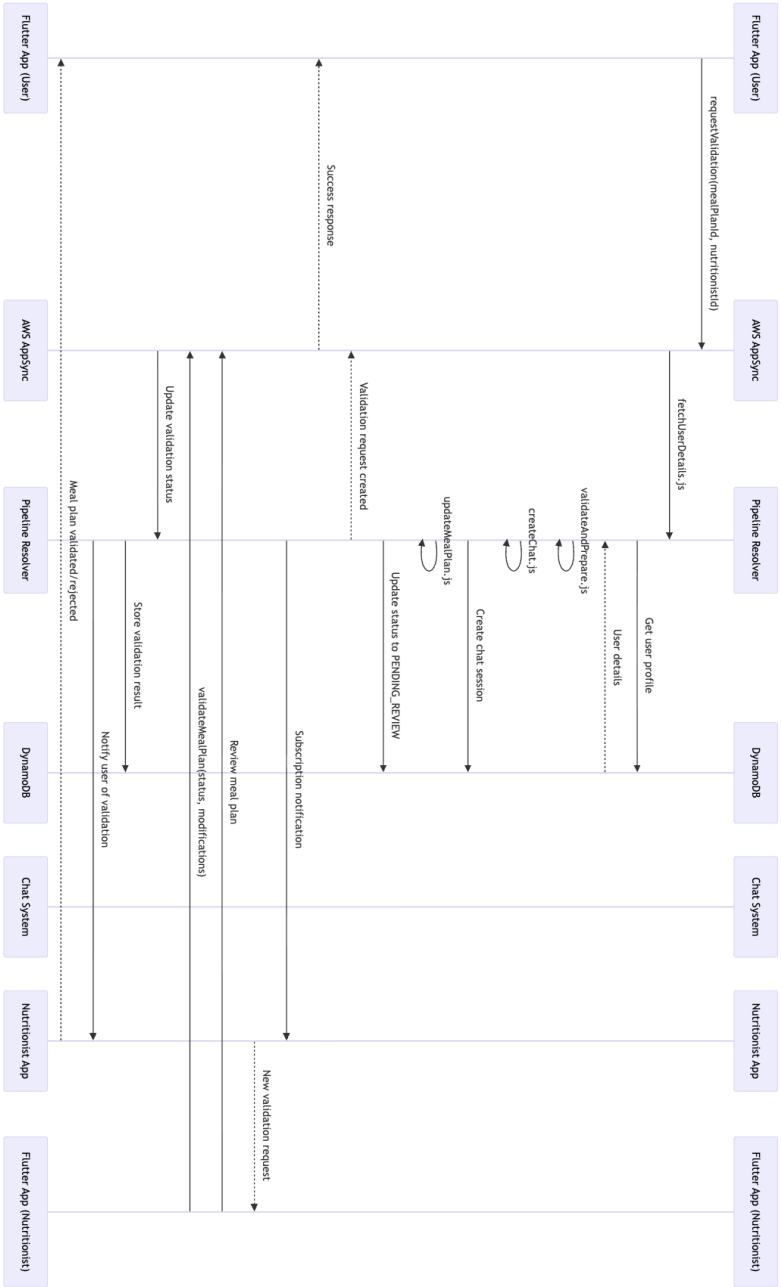


Figure 2.6: Lambda Step function to generate mealplan



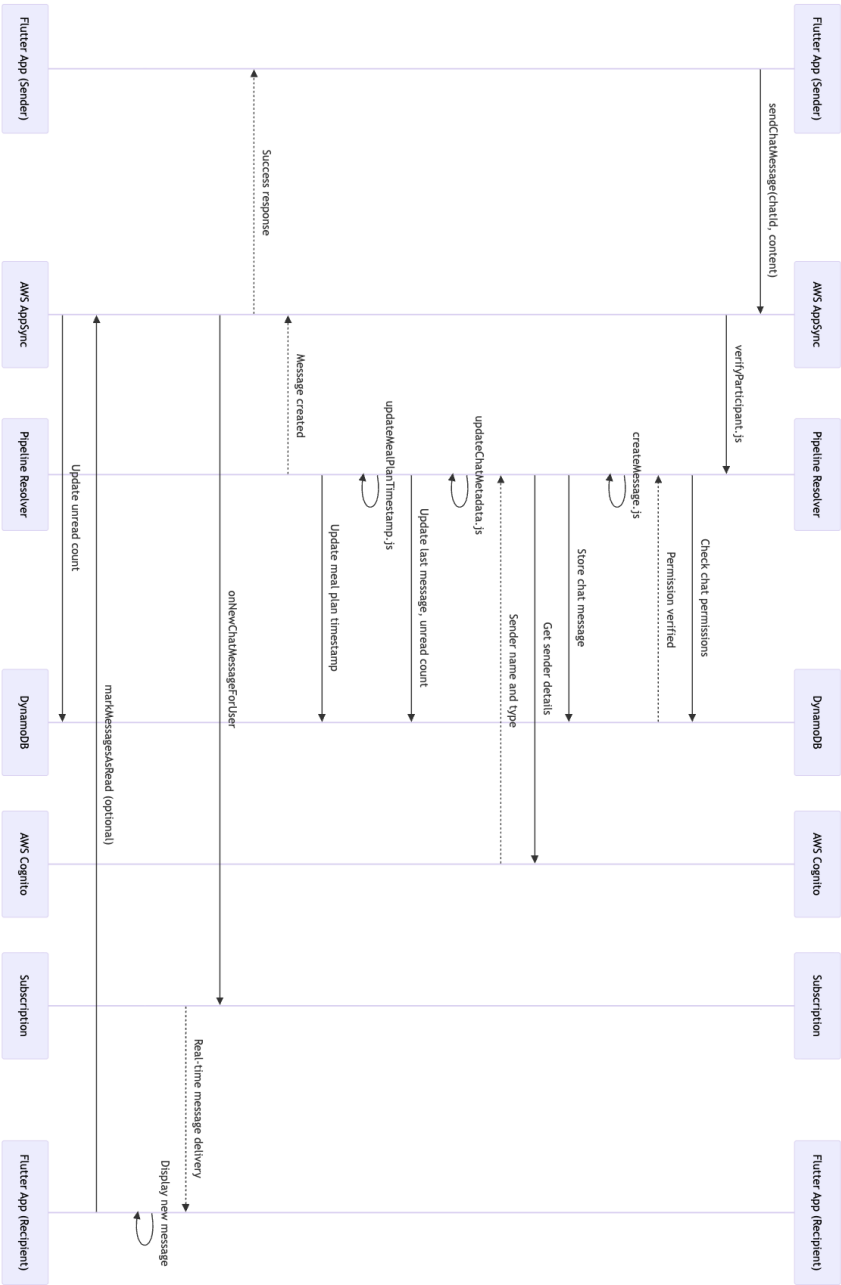
2.8.2. Nutritionist Validation Request Flow

This sequence shows how premium users request professional meal plan validation, including the creation of chat sessions and the validation workflow using pipeline resolvers.



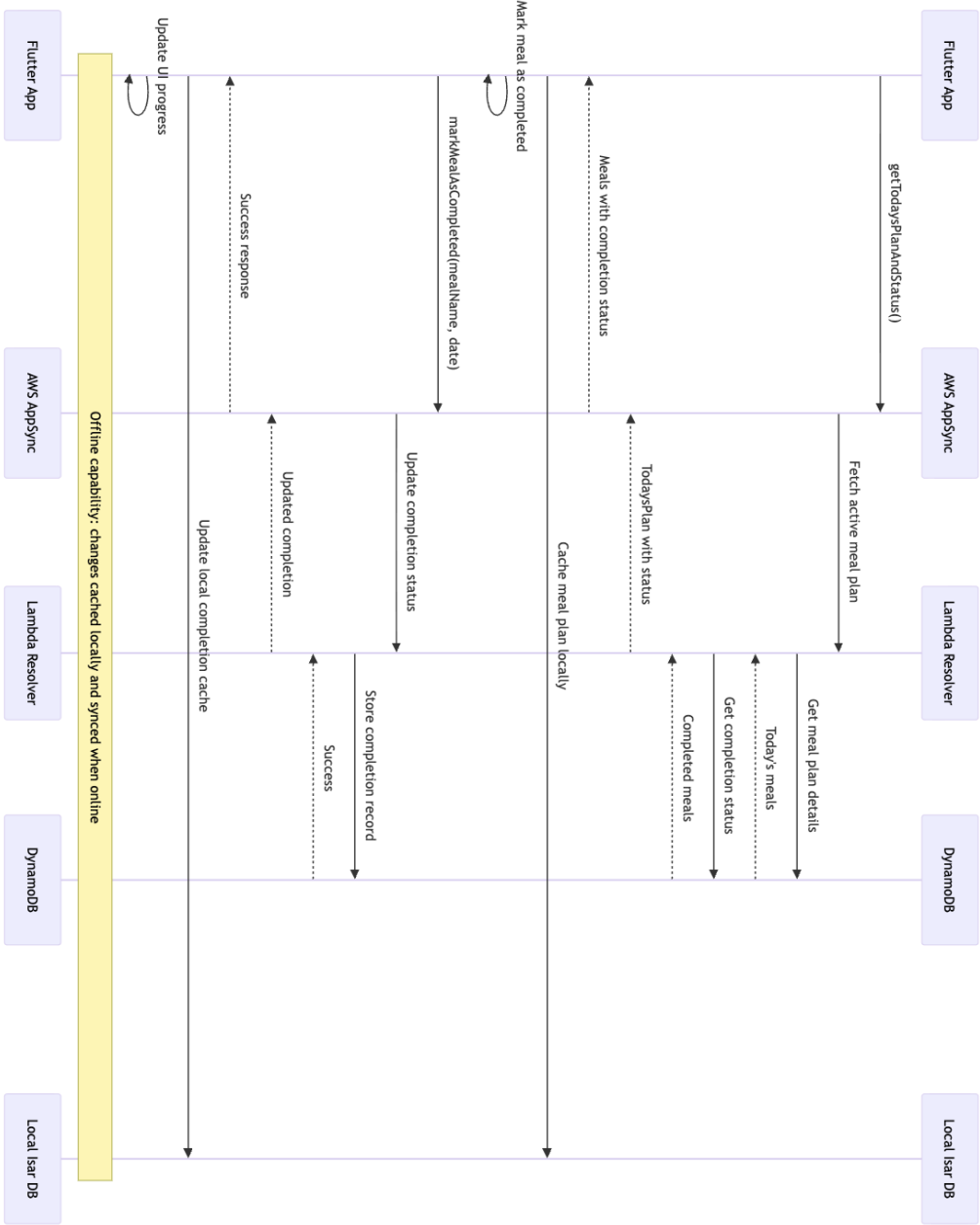
2.8.3. Real-time Chat Message Flow

This diagram illustrates the real-time messaging system between users and nutritionists, demonstrating the pipeline resolver pattern and subscription-based message delivery.



2.8.4. Daily Meal Completion Tracking Flow

This sequence explain how users track their daily meal consumption with both online and offline support through the local Isar database.



3 | User Interface (UI)

This section presents the design of the main screens of the WellPlate application. The interface is designed with role-based access, providing separate optimized experiences for regular users and nutritionists. Page layouts are responsive and adapt according to the device used to effectively exploit screen dimensions. Most screens are shown in both light and dark theme modes to demonstrate the application's theming capabilities.

3.1. UI Design Choices

3.1.1. Design Philosophy

WellPlate follows a clean, minimalist design philosophy focused on clarity and ease of use. The interface prioritizes essential information without overwhelming users. Light and dark theme options are available for personalization.

3.1.2. Navigation Pattern

The primary navigation uses a bottom navigation bar, providing quick access to main sections.

User Navigation Structure:

- Today Tab: Daily meal plan overview and tracking
- My Plans Tab: Meal plan management
- Settings Tab: Profile and preferences

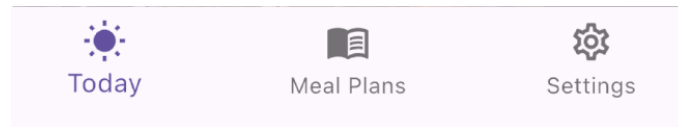


Figure 3.1: User Navigation Bar

Nutritionist Navigation Structure:

- Validate Plans Tab: Review and validation
- Settings Tab: Professional profile and availability

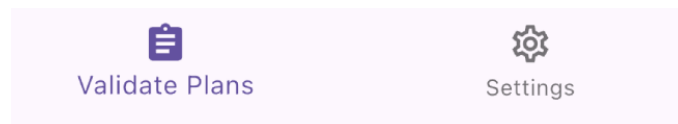


Figure 3.2: Nutritionist Navigation Bar

3.2. Smartphone UI

3.2.1. Authentication and Role Selection

The application entry point handles user authentication with role-based registration and login capabilities.

Authentication Screen Features: The authentication interface provides a streamlined onboarding experience with role selection during registration. Users can choose between regular user and nutritionist accounts, with different authentication methods available for each role. The interface includes proper form validation and error handling.

Registration Process: New users select their role (User/Nutritionist) and provide required information. Regular users can utilize Google Sign-In for convenience, while nutritionists complete detailed professional registration using email authentication.

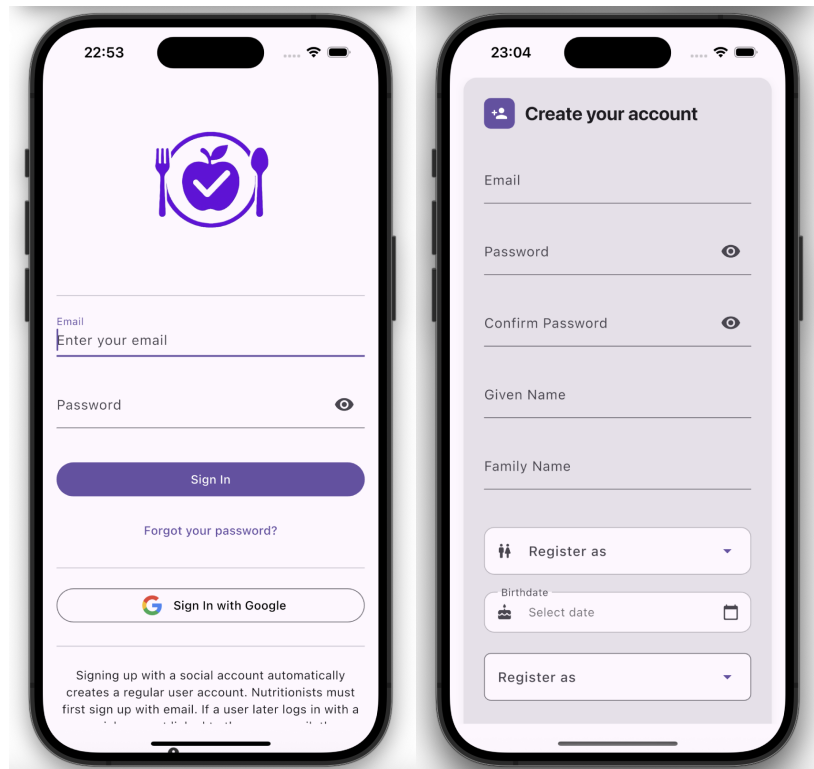


Figure 3.3: Login View

Figure 3.4: Signup View

3.2.2. User Interface – Today Tab (Daily Dashboard)

The Today Tab serves as the primary dashboard for users, providing a comprehensive view of their daily meal plan and progress tracking.

Daily Meal Overview: The main interface displays the current day's meal plan with interactive meal cards for each scheduled meal. Each card shows essential nutritional information and completion status, allowing users to track their daily progress efficiently.

Meal Card Interface: Individual meal cards present detailed information including meal names, recipe details, complete ingredient lists with quantities, and comprehensive nutritional breakdowns showing proteins, carbohydrates, fats, and total calories. Users can mark meals as completed using integrated checkboxes.

Progress Tracking: A dedicated progress card provides visual feedback on daily meal completion, helping users understand their adherence to the meal plan. The interface also includes a "Choose Plan" card when no active meal plan is selected.

State Management Display: The interface gracefully handles various application states including

loading conditions with shimmer animations, error states with retry mechanisms, empty states with clear guidance, and offline mode with cached data indicators.

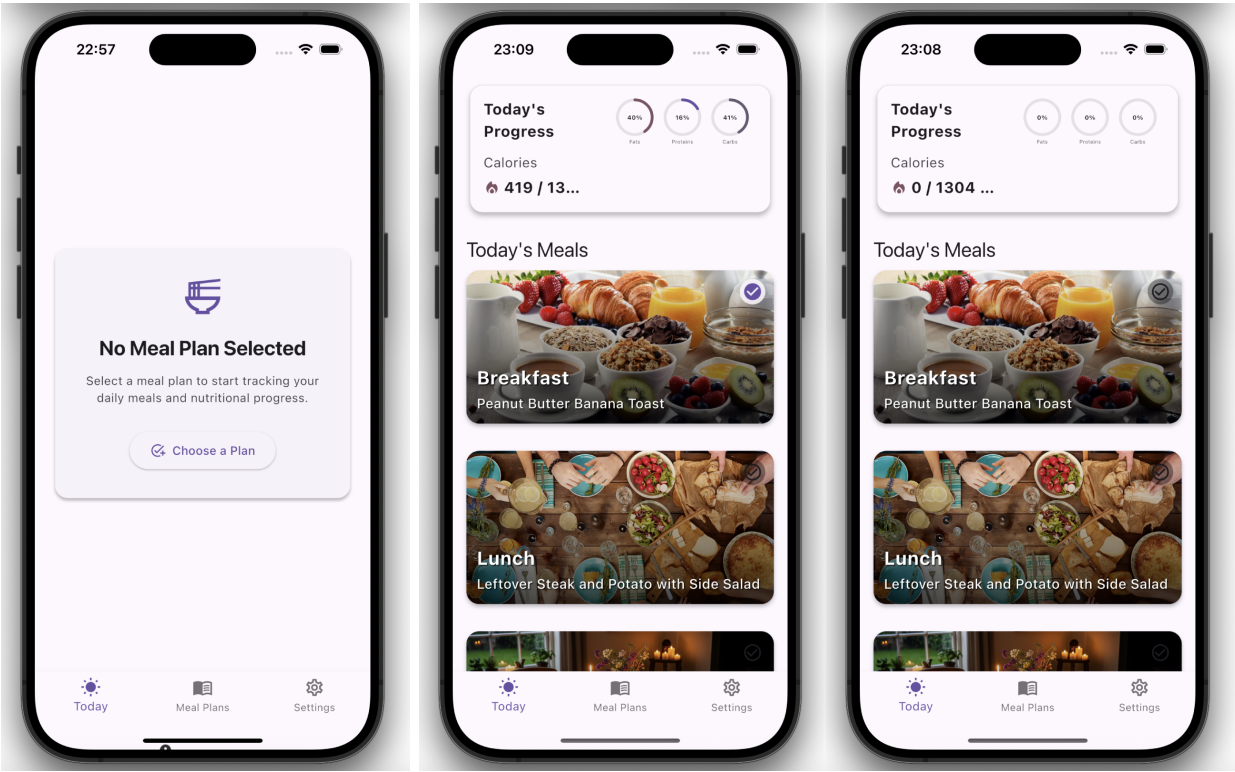


Figure 3.5: Today Page Views

3.2.3. User Interface – My Plans Tab (Meal Plan Management)

The My Plans Tab provides comprehensive meal plan management capabilities, allowing users to organize and control their nutrition planning history.

Plan List Interface: The main view displays all user meal plans in a chronological list with clear visual indicators for plan status and validation state. The currently active plan receives special highlighting to distinguish it from historical plans.

Plan Status Indicators: Each plan card shows comprehensive status information including generation status (Pending, Generated, Active, Archived, Failed) and validation status for professional review (Not Validated, Pending Review, Validated, Rejected).

Plan Management Controls: Users can perform various actions on their meal plans including setting plans as active, deleting unwanted plans, renaming plans for better organization, and requesting nutritionist validation for Pro subscribers.

Plan Details Access: Users can access detailed views of complete meal plans, showing weekly meal structures, comprehensive nutritional information, and associated metadata including creation dates and nutritionist information.

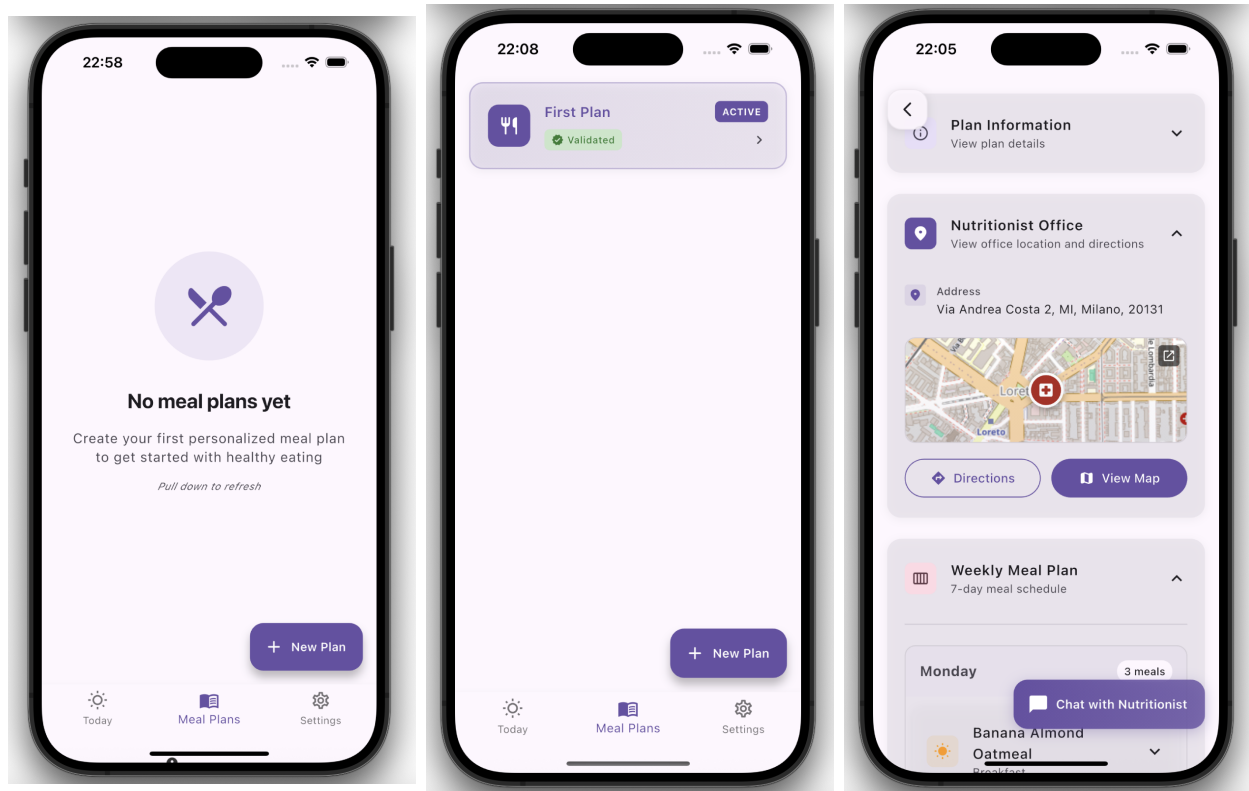


Figure 3.6: User Meal Plans view

3.2.4. User Interface – Settings Tab (Profile Management)

The Settings Tab manages comprehensive user profile information and application preferences.

User Profile Section: The interface displays user identification information including display name, email address, and profile picture with editing capabilities for personal information management.

Health Profile Management: A comprehensive form allows users to input and modify health-related information including physical attributes (weight, height), exercise frequency preferences, daily meal preferences, allergen selections from the standardized list, and free-text dietary restrictions and preferences.

Subscription Management: Pro subscription status is clearly displayed with upgrade options and benefit explanations for users considering premium features.

Security and Account Settings: Users can access password change functionality and account management options including data deletion controls in a clearly marked danger zone.

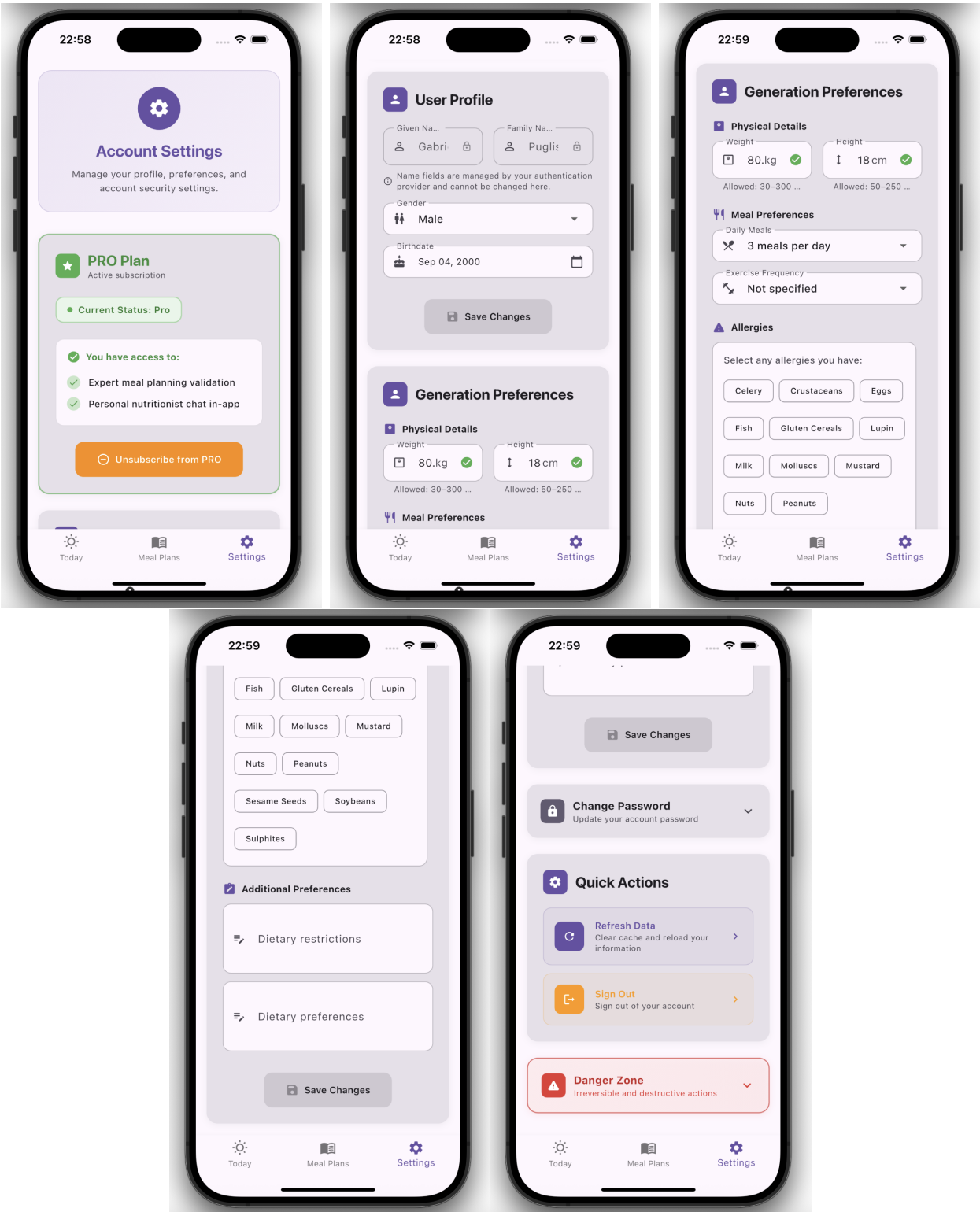


Figure 3.7: User Setting View

3.2.5. Nutritionist Interface – Validate Plans Tab (Professional Dashboard)

The Validate Plans Tab serves as the primary workspace for nutrition professionals, providing comprehensive tools for meal plan review and client management.

Validation Queue Interface: The main view displays pending meal plan validation requests in an organized queue, allowing nutritionists to efficiently prioritize and manage their client review workload.

Client Plan Review Cards: Each validation request is presented as a comprehensive card showing user health profiles, complete meal plan information, nutritional analysis, and request priority indicators to help nutritionists make informed decisions.

Professional Validation Tools: The interface provides professional-grade controls for meal plan approval, rejection with detailed feedback options, and modification requests. Nutritionists can access complete client health information and integration with the chat system for direct communication.

Client Management Dashboard: Nutritionists can access comprehensive client overviews, validation history, and professional notes to maintain continuity in their consultation services.

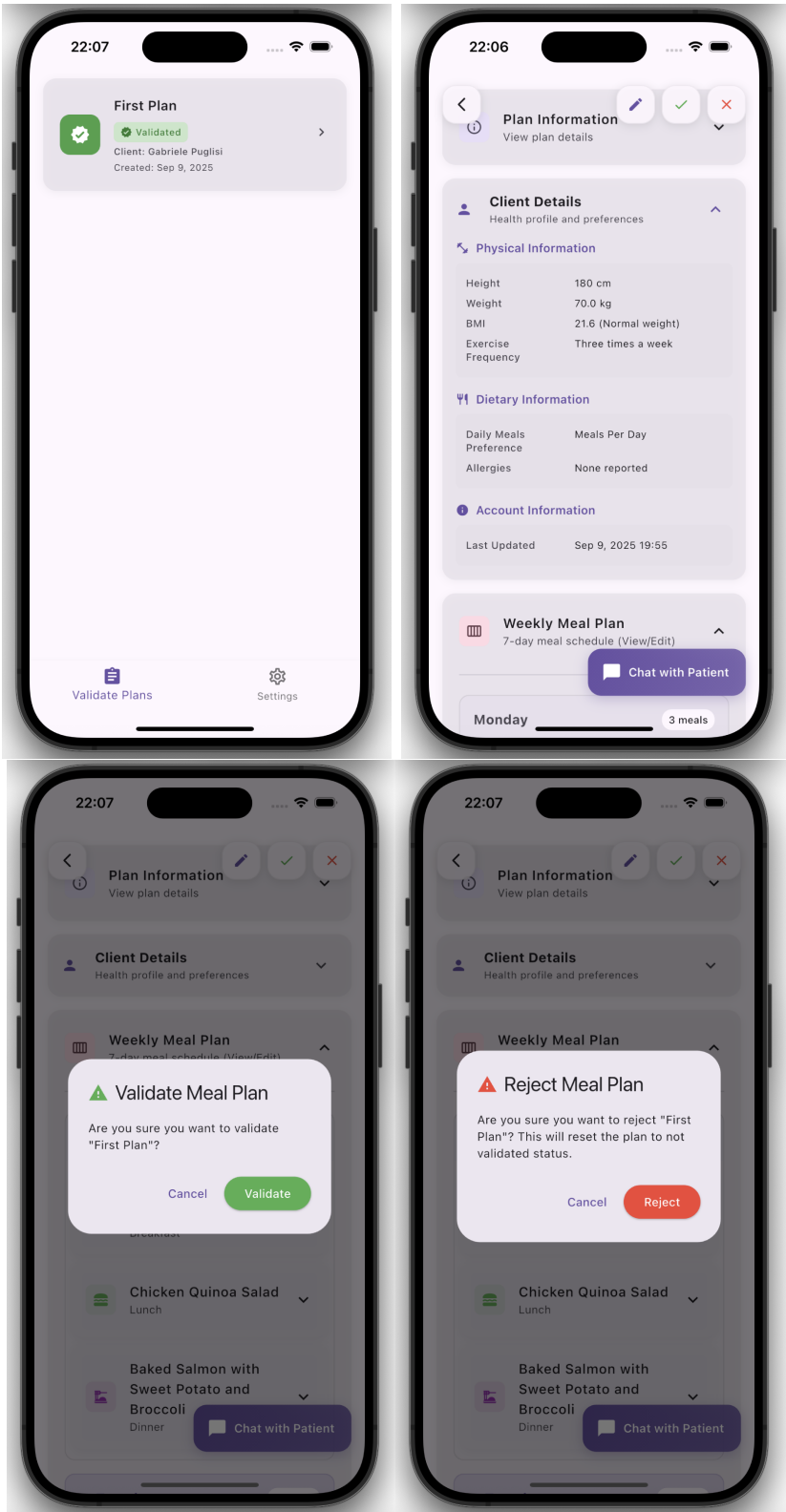


Figure 3.8: Validate Plans view

3.2.6. Nutritionist Interface – Settings Tab (Professional Profile)

The Nutritionist Settings Tab manages professional profile information and service availability.

Professional Profile Management: The interface allows nutritionists to manage their professional information including specialization areas, professional bio and credentials, profile picture management, and availability status for accepting new client consultations.

Service Management Tools: Nutritionists can access client management tools, consultation history, and professional account settings to maintain their practice efficiently.

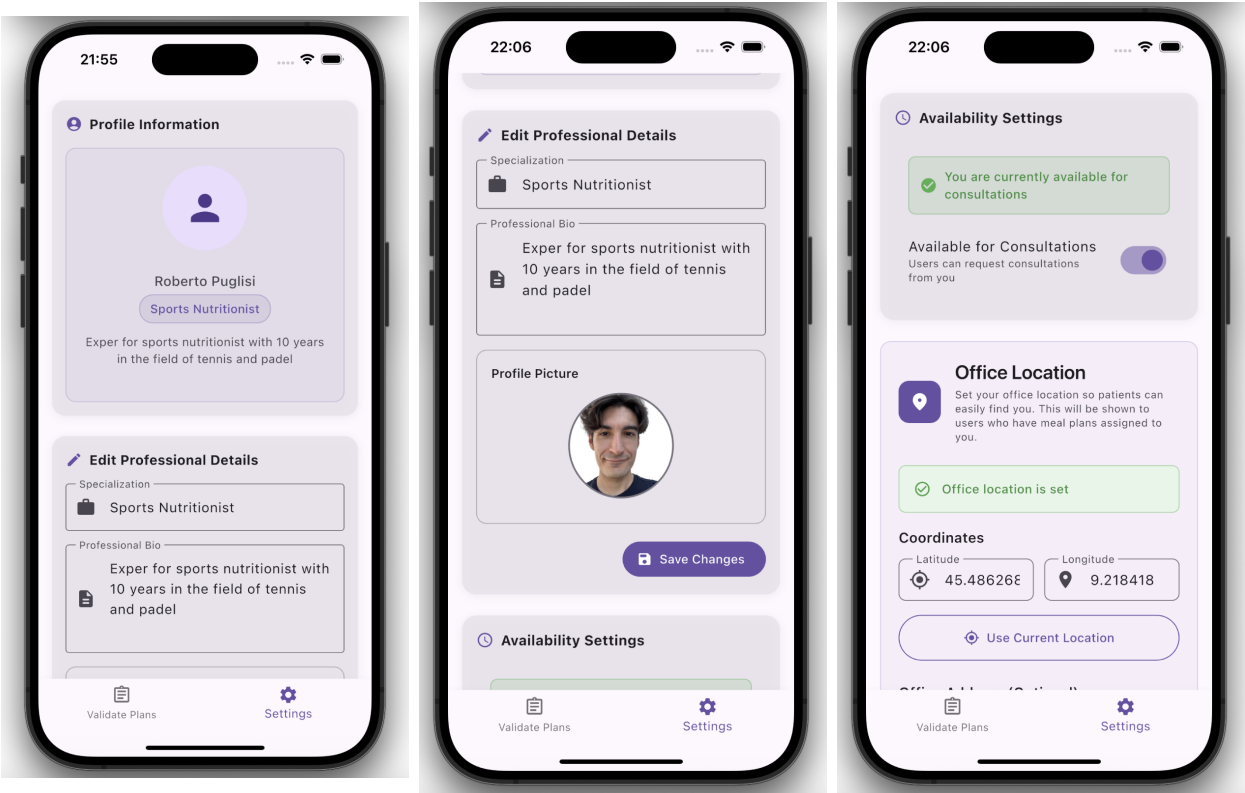


Figure 3.9: Nutrionist Setting View

3.2.7. Chat System (Shared Communication Interface)

The integrated chat system provides real-time communication capabilities between users and nutritionists.

Chat List Interface: Users and nutritionists can access their conversation lists with clear indicators for unread messages, conversation participants, and recent message previews.

Message Interface: The chat interface provides real-time messaging with distinct visual styling for users versus nutritionists, comprehensive message history with pagination, and rich text input capabilities.

Integration Features: The chat system integrates seamlessly with meal plan validation workflows and includes cross-application notification management for timely communication.

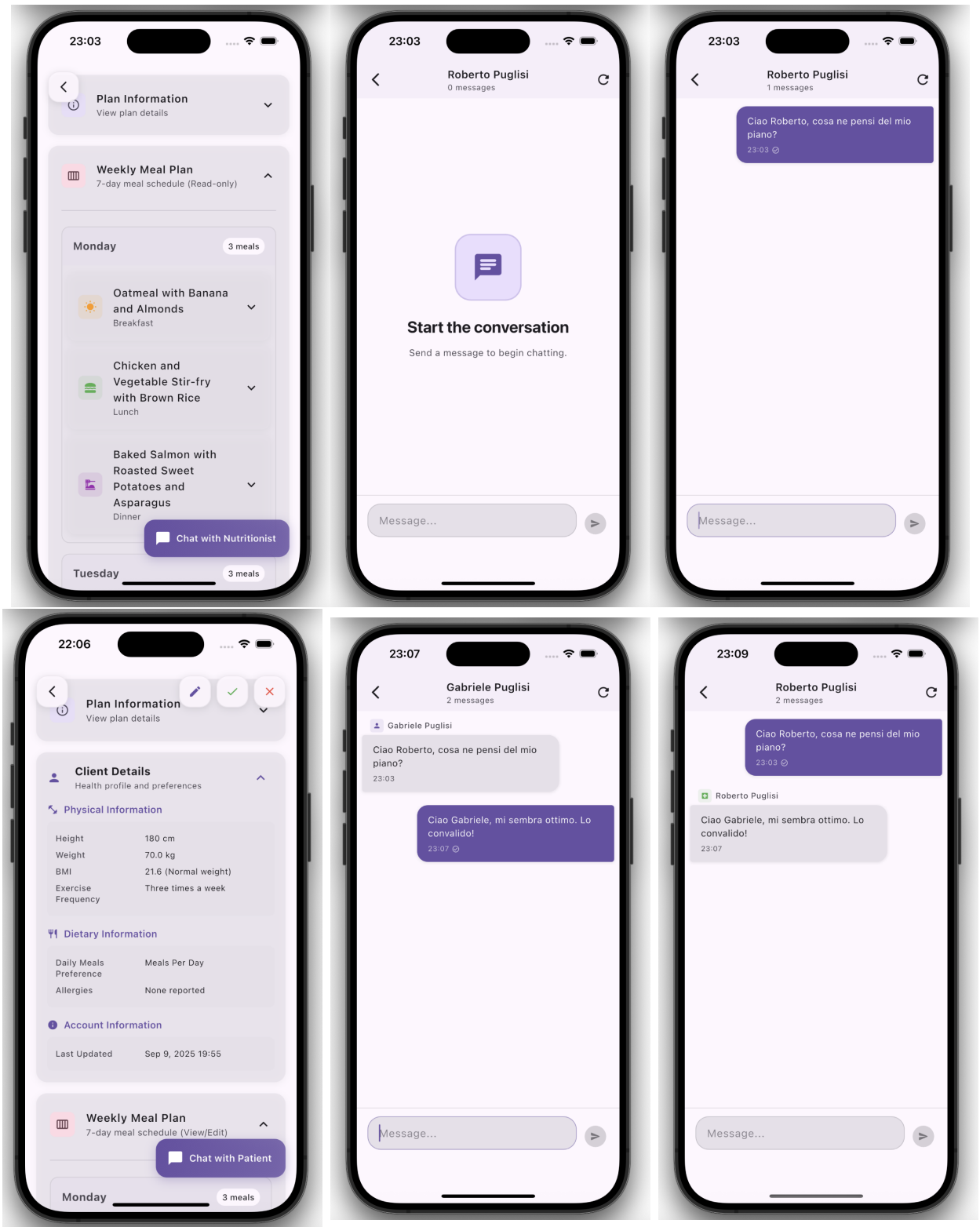


Figure 3.10: Nutrionist and User chat views

3.2.8. Meal Plan Generation Interface

The meal plan generation process provides guided AI-powered meal plan creation with user preference customization.

Generation Configuration: Users can specify preference overrides for individual meal plans, allowing customization beyond their standard profile settings for specific planning periods.

Progress Monitoring: Real-time generation progress is displayed with status updates delivered via GraphQL subscriptions, keeping users informed throughout the AI processing workflow.

Result Presentation: Successful generation presents users with meal plan previews, while errors are handled and explained in the UI.

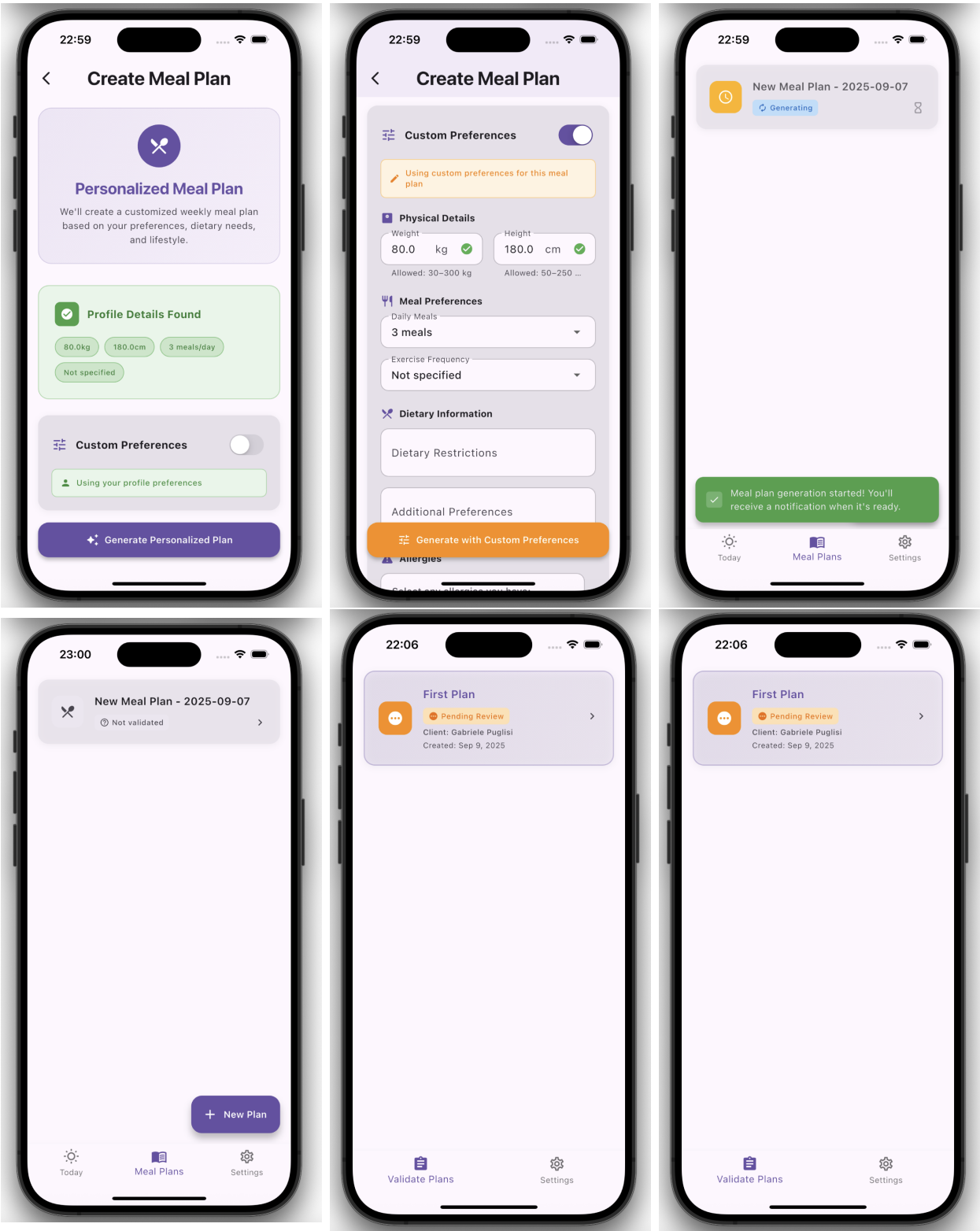


Figure 3.11: Meal plan generation and validation views

3.2.9. Detailed Meal and Plan Viewing

Comprehensive viewing interfaces provide detailed access to meal and plan information.

Meal Details Interface: Individual meal views present complete ingredient lists with precise quantities, detailed nutritional breakdowns, step-by-step recipe instructions, and preparation guidance.

Complete Plan Reading: Full meal plan interfaces show weekly overviews, daily nutritional summaries, comprehensive ingredient shopping lists, and complete plan metadata including nutritionist information and validation status.

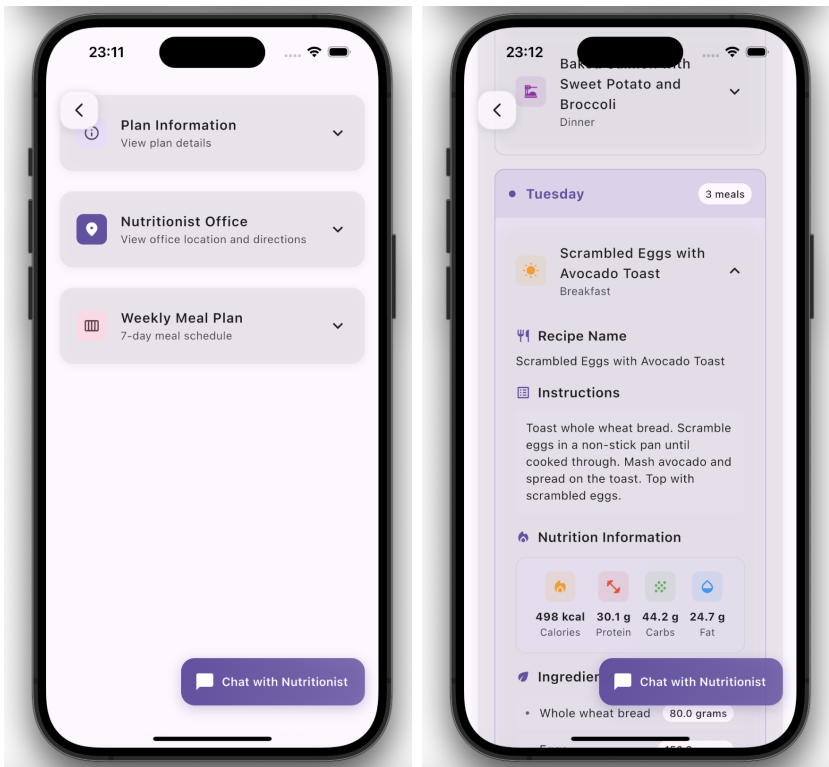


Figure 3.12: Meal plan details views

3.2.10. Tablet UI

The tablet interface automatically detects larger screens to implement distinct layouts optimized for tablet dimensions while maintaining the same functions of the smartphone versions.

Adaptive Layout Strategy

- **Screen Detection:** The application automatically identifies tablet-sized screens to trigger optimized layouts, ensuring appropriate experiences across device types.
- **Multi-Panel Architecture:** Tablet layouts transition from single-column smartphone designs to multi-panel arrangements that display related information simultaneously, reducing the navigation through different pages.
- **Enhanced Spacing System:** Tablet interfaces use more padding and margins to create comfortable whitespace that takes advantage of the larger screen

Today Tab Adaptations

- **Landscape Mode:** Implements two-column layout with progress card positioned alongside the meals section, creating a dashboard-style interface that maximizes horizontal space utilization.
- **Enhanced Information Density:** Meal cards display additional nutritional details and larger touch targets.

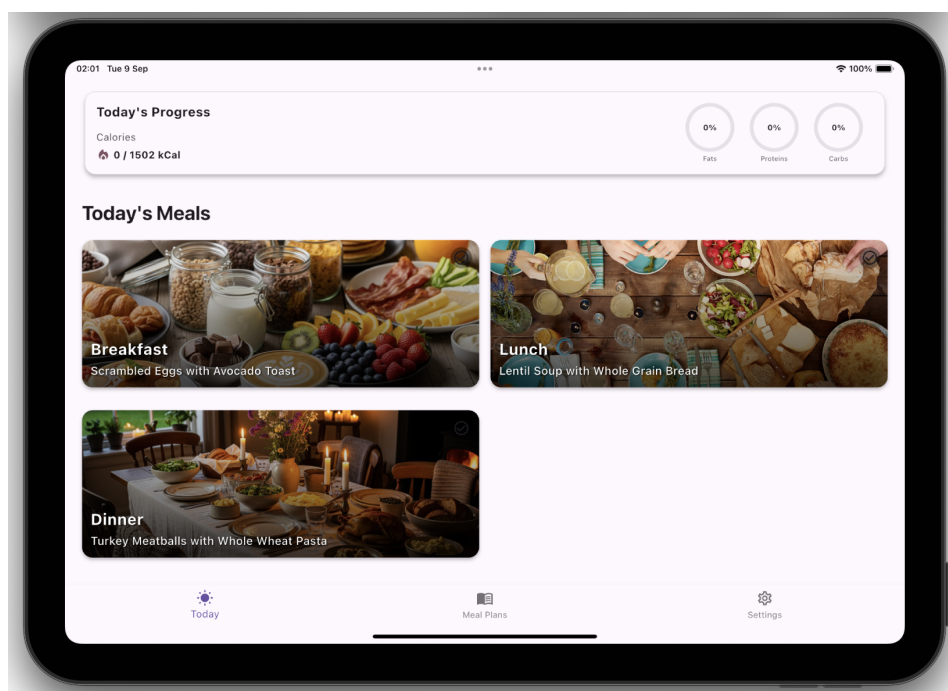


Figure 3.13: Today Tab Tablet view

Plans Tab

- **New dashboard view:** Meal plans display in a responsive grid instead of vertical list format, enabling quick visual scanning of plan library with column count adapting to available width.
- **Persistent Detail Panel:** Selected meal plans show complete details in a dedicated right panel, eliminating the need for separate navigation screens and maintaining context during plan comparison.
- **Visual Selection States:** Clear indication of selected plans in the grid with contextual actions accessible through long-press menus rather than swipe gestures.

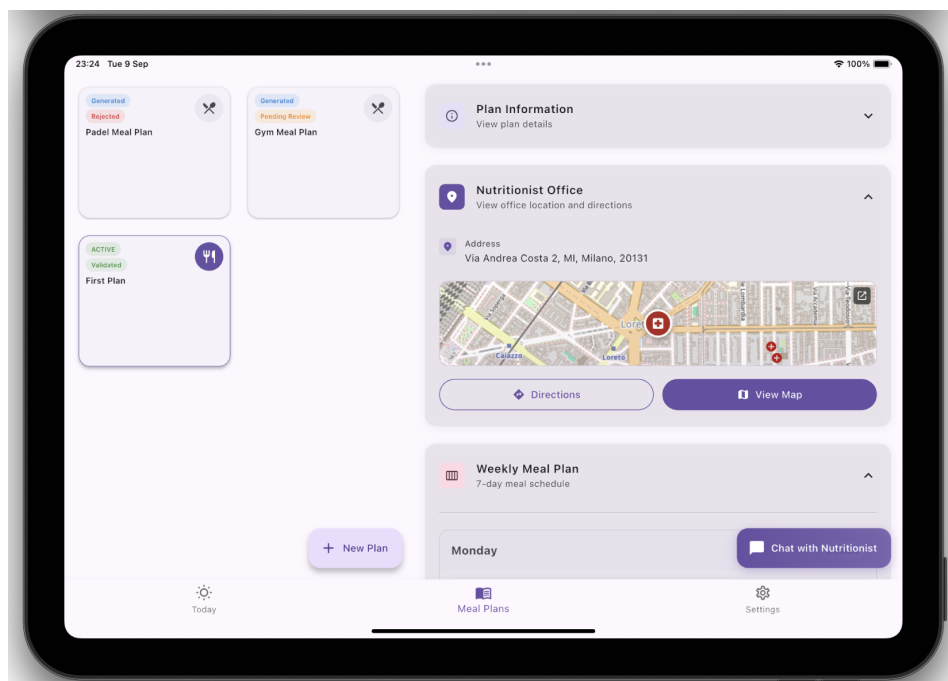


Figure 3.14: Meal Plans Tab view

Nutritionist Interface Enhancements

- **New Dashboard View:** Validation interface simultaneously displays pending validations, detailed plan review, and client communication panels, supporting efficient professional workflow without context switching.

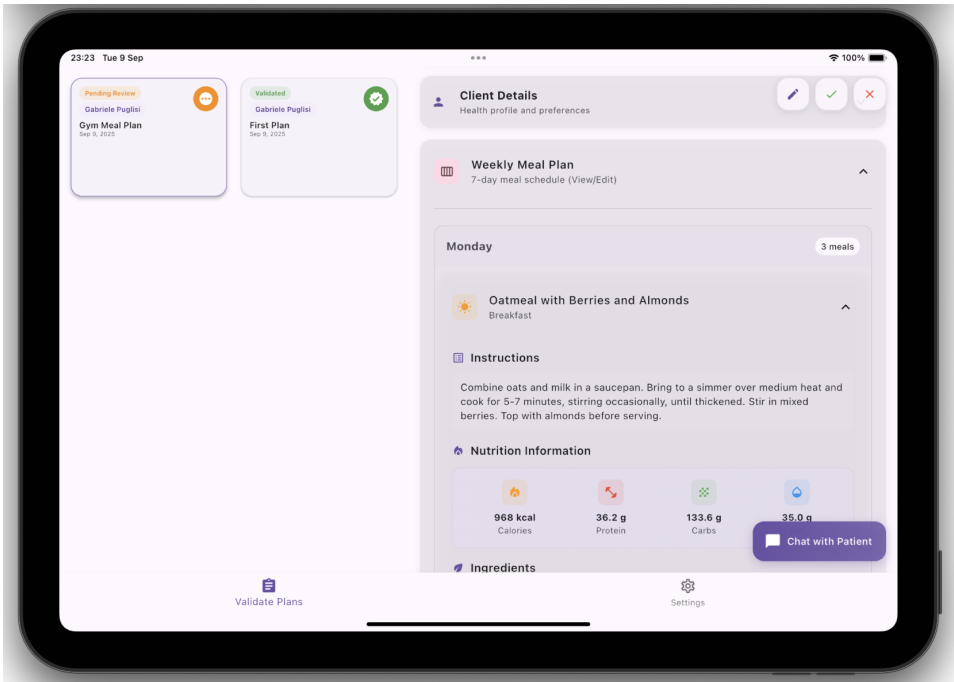


Figure 3.15: Nutritionist Validate Plans Tab

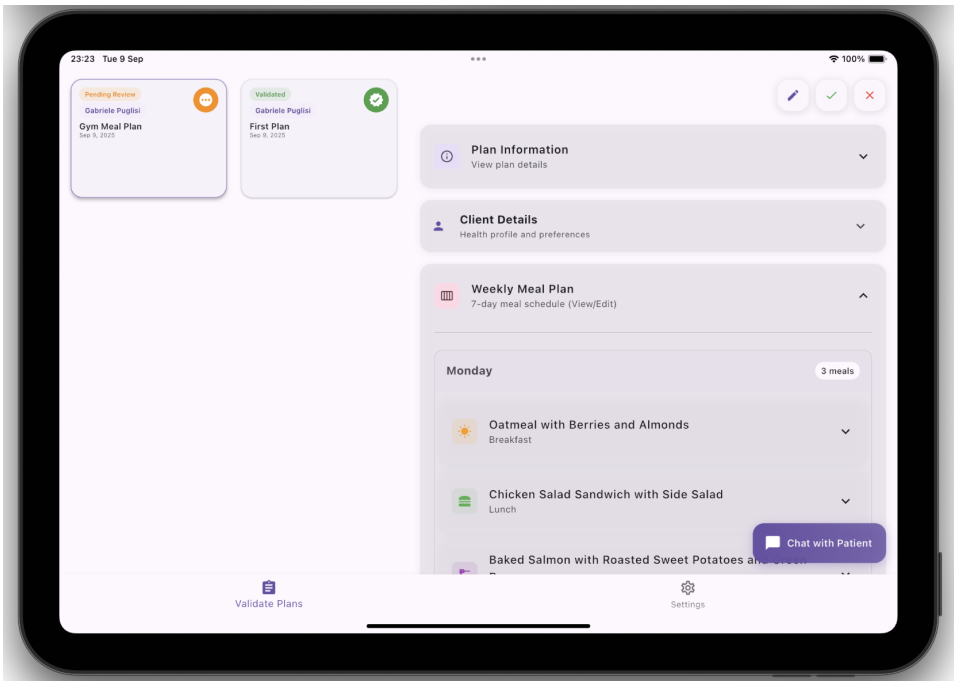


Figure 3.16: Meal Plan Details

3.3. Accessibility

WellPlate includes accessibility features to ensure the application serves users with diverse needs and preferences, supporting inclusive nutrition planning across different users.

- **Visual Accessibility:** The application provides both light and dark theme options to accommodate users with different visual preferences and sensitivities. Theme selection is persistent across app sessions and applies consistently throughout all interface elements.
- **Language Support:** Multi-language functionality includes Italian and English translations, enabling users to interact with the application in their preferred language.



Figure 3.17: Dark theme

Figure 3.18: App in Italian

4 | Testing Campaign

4.1. Testing Environment

The tests for *WellPlate* are performed in two environments:

- **Mock Environment:** Main external APIs (AWS AppSync, AWS Cognito, AWS Storage) are replaced with mock versions using static test data. This allows testing without external service dependencies and is implemented using Riverpod provider overrides with mock implementations. To achieve that we leveraged the Dependency Injection design pattern.
- **Development Environment:** Uses real AWS services with test data to validate complete functionality including AI meal generation, authentication, and real-time chat features.

4.2. Unit Testing

Unit tests focus on core Flutter functionality that can be tested in isolation:

- **Data Model Conversion:** Testing conversion between Amplify models and local cache models
- **State Management:** Validating Riverpod provider behavior with mocked dependencies
- **Utility Functions:** Testing nutritional calculations, date formatting, and input validation
- **Business Logic:** Testing meal plan status transitions and validation rules

Due to the serverless architecture where most business logic resides in AWS Lambda functions, client-side unit testing focuses on data handling and UI state management.

4.2.1. Coverage Report

The following is the cover report from unit testing.

File	Lines	Line Coverage
- All Files	2753/3089	89.1%
- lib	2753/3089	89.1%
+ models	550/562	97.9%
+ providers	838/926	90.5%
+ services	1251/1474	84.9%
+ Utils	114/127	89.8%

It is easy to notice how coverage is slightly higher for model and utility classes, while it is lower (but still satisfactory) for service and provider classes which are more cumbersome to test due to the need of many mocked network calls and inner complexity.

From unit testing we excluded all the automatically generated classes since they are generated from already tested libraries.

4.3. User Testing

User testing focuses on getting authentic feedback from potential users to understand how well *WellPlate* meets real-world scenarios and to identify areas for improvement.

4.3.1. Testing Approach

Participants: 6 individuals including potential app users interested in nutrition planning and 2 people with some nutrition knowledge or experience.

Session Format:

- **App Exploration (20 minutes):** Participants use the app freely to accomplish nutrition-related goals they would naturally have.
- **Open Discussion (20 minutes):** Conversation about their experience, challenges, and suggestions.

4.3.2. Key Questions for Feedback

Rather than structured tasks, testing focuses on understanding user perspectives through open-ended discussion. The following questions were used:

1. What was your first impression when opening the app?
2. Which features did you find most/least useful for your nutrition goals?
3. What confused you or felt unclear during your usage?
4. How does this compare to how you currently plan meals or track nutrition?
5. What would make you want to use this app regularly?
6. What features are missing that you would expect in a nutrition app?
7. Would you trust the AI-generated meal plans for your dietary needs?
8. How do you feel about the nutritionist consultation feature?

From the user testers' responses, we understood that in general both users and nutritionists felt they could navigate the app pretty easily. Sometimes, they took some more time to understand all the features, but in the end they all managed to accomplish their intended tasks in a reasonable time. In general, we collect positive feedback on the possibility of using *WellPlate* in everyday life even though two people were skeptical about relying on an LLM-generated meal plan.

5 | Future Works

WellPlate's current implementation provides a solid foundation for personalized nutrition planning and professional consultation services. The modular architecture and serverless backend design facilitate straightforward expansion of functionality without requiring significant infrastructure changes. Several enhancement opportunities have been identified that would extend the application's capabilities and user value.

5.1. Improve AI integration

The meal plan generation system could benefit from more sophisticated AI integration. Advanced machine learning models could analyze user eating patterns and preferences over time to provide increasingly personalized recommendations. Integration with computer vision APIs would enable users to photograph meals for automatic nutritional analysis and plan adherence tracking. Natural language processing capabilities could enhance the chat system with automated nutrition education responses and preliminary meal planning suggestions.

5.2. Wearable Device Integration

Connecting with fitness trackers and health monitoring devices would provide real-time data for more accurate meal plan adjustments. Integration with popular platforms like Apple Health, Google Fit, and Fitbit would enable automatic exercise tracking, calorie burn calculations, and biometric monitoring that could inform daily nutritional recommendations.

5.3. Social and Community Features

A community platform where users can share meal photos, recipes, and success stories would increase engagement and motivation. Group challenges, shared meal plans among families or friends, and

peer support systems could enhance the social aspects of nutrition planning. Rating and review systems for nutritionist services would help users select appropriate professional support.

5.4. Advanced Nutritionist Tools

Professional features could include comprehensive client progress analytics, customizable meal plan templates and integration with existing management software.

5.5. Technical Improvements

Performance optimizations including improved caching strategies and improved offline functionality.